A Project Report on

"Smart Triage System Using NLP and ML"

Submitted to

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY ANANTAPUR

In partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering(AI)

By

SHAIK AFREEN	**222T1A3152**
SHAIK SABIHA BANU	**222T1A3153**
SHAIK SUHANA	**222T1A3154**
SYED AAISHA	**222T1A3157**

Under the esteemed guidance of

Mr.M. CHINNA RAJU **M.Tech, (P.hd)**

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING -AI

ASHOKA WOMEN'S ENGINEERING COLLEGE

(Approved by AICTE, NEW DELHI & Affiliated to JNTUA Anantapur)

An ISO 9001:2000 Certified Institution

OPP.DUPADU (RS), NH-44, LAKSHMIPURAM (PO), KURNOOL518218. AP.

2022-2026

STUDENT DECLARATION

We hereby declare that the project work and entitled "SMART TRIAGE SYSTEM USING NLP AND ML" submitted by us for the award of Degree of Bachelor of Technological University Anantapuram and is a bonafied record of work done in ASHOKA WOMEN'S ENGINEERING COLLEGE, KURNOOL and has not been submitted to any other University for award of any degree.

Date:

Place: Kurnool Signature of the student.

GUIDE DECLARATION

I hereby declare that the project work and entitled "SMART TRIAGE SYSTEM USING NLP AND ML" done by SHAIK AFREEN (222T1A3152), SHAIK SABIHA BANU (222T1A3153), SHAIK SUHANA (222T1A3154), SYED AAISHA (222T1A3157) under my guidance.

Date:

Place: Kurnool

Signature of the Guide.

Ph:08518-287619
Fax:08518-287618

ASHOKA WOMEN'S ENGINEERING COLLEGE

An Engineering College Sponsored by

Vishwam Educational Society, Kurnool

Approved by AICTE, New Delhi and Affiliated to JNTUA, Anantapur
Opp.Dupadu (RS), N.H-44, Kurnool -518218, Kurnool District, A.P.
www.ashokacollege.in

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING(AI)

BONAFIDE CERTIFICATE

This is to certify that the Project Report Entitled "SMART TRIAGE SYSTEM USING NLP AND ML" is the bonafied work done by SHAIK AFREEN (222T1A3152), SHAIK SABIHA BANU (222T1A3153), SHAIK SUHANA (222T1A3154), SYED AAISHA (222T1A3157) in the Department of Computer Science and Engineering ASHOKA WOMEN'S ENGINEERING COLLEGE, Kurnool in the partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering(AI) from Jawaharlal Nehru Technological University Anantapur during the academic year 2025-2026.This work has been carried out under my guidance.

GUIDE	**HOD**
Mr. M.CHINNA RAJU	**DR. SYED SHOEB PEER**
M.Tech, (P.hd)	**M.Tech, P.hd, LLB**
Assistant Professor	**Associate professor**

Place:

Date:

Certificate that the candidates were examined by the viva-voce Examination held at ASHOKA WOMEN'S ENGINEERING COLLEGE, Kurnool on

Signature of Internal Examiner **Signature of External Examiner**

ACKNOWLEDGEMENT

I would like to express my thanks to many of the people. I can list here few of them are **Mr.M.CHINNA RAJU,** Assistant Professor, M.Tech, (P.hd), Dept. of CSE, my project guide from him we learnt many things about the subject which are very helpful to complete my project and he guided me in right direction.

We express our gratitude to **SYED SHOEB PEER,** M.Tech, P.hd, LLB HoD Dept. of CSE-AI, for the project facilities made available to us in the department and he supported me throughout my project period.

At the outset we thank our honourable Correspondent **Sri K.ASHOK VARDAN REDDY** Garu**,** and our beloved Principal **Dr. R. NAVEEN**M.E, **Ph.D.,** for providing us with good faculty and making their moral support throughout the course. Finally, we extend our sincere thanks to all the **Staff Members** of CSE Department(AI) who have cooperation and encouraged us in making our project successful.

We owe our thanks and deep appreciation much more than words can express to my **parents** and family members without their cooperation, constant support and encouragement this would have been a distant dream.

By:

SHAIK AFREEN	**222T1A3152**
SHAIK SABIHA BANU	**222T1A3153**
SHAIK SUHANA	**222T1A3154**
SYED AAISHA	**222T1A3157**

ABSTRACT

The Smart Triage System is an AI-powered application designed to classify patient conditions based on textual descriptions of symptoms using Natural Language Processing (NLP) and Machine Learning (ML) techniques. By analyzing patient-reported data, the system predicts the urgency of medical attention required, categorizing cases into high, medium, or low triage levels. This automation aims to assist healthcare providers in prioritizing patients efficiently, ensuring that critical cases receive immediate attention while reducing unnecessary delays for routine conditions.

The system leverages a pre-trained BERT (Bidirectional Encoder Representations from Transformers) model, fine-tuned on a triage-specific dataset, enabling it to understand complex clinical language and context. Using advanced tokenization and embedding techniques, the model captures nuanced patterns in patient descriptions, allowing accurate classification even when symptom descriptions are unstructured or incomplete. Additionally, the system provides probability scores for each triage level, offering transparency and confidence metrics to clinicians.

An important aspect of the system is its explainability. The platform highlights key symptom keywords from the input text and provides natural-language explanations for the predicted triage level. For instance, high-urgency predictions are accompanied by alerts indicating that immediate medical attention is required, while medium and low-risk cases include guidance for clinician verification or routine monitoring. This interpretability ensures trust in the system's recommendations and facilitates informed decision-making by healthcare professionals.

The Smart Triage System has significant implications for healthcare operations, especially in emergency departments and telemedicine platforms. By combining NLP with ML, it streamlines patient assessment, improves resource allocation, and reduces human error in initial triage. Its integration into clinical workflows promises faster patient prioritization, better outcomes, and overall efficiency in healthcare delivery, demonstrating the transformative potential of AI in modern medical practice.

TABLE OF CONTENTS

S.NO	TOPICS	PAGE NO
	ABSRACT	I
Chapter :1	**INTRODUCTION**	
	1.1 General	1
	1.2 Motivation	2-3
	1.3 Problem Definition	4-5
	1.4 Limitations of the project	5
	1.5 Organization of the project	6
Chapter: 2	**LITERATURE SURVEY**	
	2.1 Introduction	7
	2.2 Literature review	7-8
	2.3 Existing System	9-10
	2.4 Drawbacks of Existing System	10-11
	2.5 System Requirements	11
Chapter: 3	**PROPOSED SYSTEMS**	
	3.1 Introduction	12
	3.2 Proposed system	12-15
	3.3 Algorithm Used	15-16
	3.4 Advantages of Proposed System	16-17
Chapter:4	**METHODOLOGY**	
	4.1 Introduction	18
	4.2 Anaconda	19
	4.3 Libraries Used	19-20

Chapter: 5	**SOURCE CODE**	
	5.1 Introduction	21-22
	5.2 Predicted Model	23
	5.3 Training Model	24-26
	5.4 Applications	26-27
Chapter:6	**EXPERIMENTAL RESULTS**	
	6.1 Run the app	28
	6.2 Output	29
Chapter:7	**RESULT ANALYSIS**	
	7.1 Result Analysis	30-31
Chapter:8	**CONCLUSIONAND FUTURE ENHANCEMENT**	
	8.1 Conclusion	32
	8.2 Future scope	33-34
Chapter:9	**References**	35

LIST OF FIGURES

S.NO	FIGURE NAME	PAGE NO
1	NLP-Based Symptom Interpretation	13
2	BERT Model for Classification	13
3	Flask Web Application Setup	22
4	Training process flow chart	26

CHAPTER: 1

INTRODUCTION

1.1 GENERAL

Healthcare systems, especially emergency departments, often face challenges in efficiently assessing and prioritizing patients based on the severity of their conditions. Traditional triage methods rely heavily on manual evaluation by medical staff, which can be time-consuming, inconsistent, and prone to human error. Delays in identifying high-risk patients may lead to adverse outcomes, while over-triaging low-risk cases can strain medical resources. With the rapid growth of patient data, especially in telemedicine and remote consultations, there is a pressing need for automated, reliable, and accurate triage solutions.

The Smart Triage System addresses this challenge by employing Natural Language Processing (NLP) techniques to interpret patient-reported symptoms and combine them with Machine Learning (ML) models to predict the urgency of care. Patient descriptions, often unstructured and varied in terminology, are transformed into meaningful numerical representations using BERT embeddings, capturing context, relationships between symptoms, and clinical nuances. This allows the system to classify triage levels—high, medium, or low—effectively, even with limited or noisy input data.

The system integrates probability scores, confidence warnings, and keyword highlights to provide a comprehensive view of each prediction. By quantifying model certainty and identifying influential words, clinicians are empowered to make informed decisions, reducing reliance solely on automated outputs. Additionally, the approach supports interpretability, which is crucial in medical applications, where understanding the rationale behind predictions is as important as the prediction itself.

Motivated by the need to improve patient outcomes, optimize emergency department workflow, and reduce cognitive load on medical staff, this project leverages AI to augment human decision-making rather than replace it. By automating initial triage assessments and providing actionable insights, the Smart Triage System aims to enhance healthcare efficiency, minimize risk to patients, and streamline the delivery of timely medical care across diverse clinical settings.

1.2 MOTIVATION:

The motivation behind this project stems from several key factors:

1. Reducing Delays in Emergency Care:

One of the main motivations behind developing the Smart Triage System is the need to reduce delays in emergency departments. Manual triage can be slow due to high patient volumes, leading to critical patients waiting longer for attention. Automating the initial assessment using NLP and ML ensures that high-risk cases are flagged immediately, allowing medical staff to focus on urgent patients and improving survival rates.

2. Handling Unstructured Patient Data:

Patient symptom descriptions are often unstructured, containing varied terminology, abbreviations, and incomplete information. Traditional systems struggle to interpret such input consistently. Using NLP models like BERT allows the system to understand the semantic meaning behind patient-reported symptoms, capturing nuances that simple keyword-based or rule-based systems would miss.

3. Supporting Overburdened Medical Staff:

Healthcare professionals, especially in emergency settings, face high workloads that can impact decision-making. The Smart Triage System assists by providing preliminary triage suggestions, confidence scores, and explanations, reducing cognitive load and allowing clinicians to prioritize their attention more effectively.

4. Improving Consistency and Accuracy:

Human triage can vary based on experience, fatigue, and subjective judgment. Automating the triage process ensures consistency in assessing similar cases. The ML model, trained on historical patient data, can reliably classify triage levels, minimizing errors due to human bias or oversight targeting results in higher conversion rates and better monetization opportunities for publishers and advertisers.

5. Facilitating Telemedicine and Remote Healthcare:

With the rise of telemedicine, patients increasingly submit symptom descriptions online. The Smart triage System enables remote assessment, providing healthcare providers with immediate triage suggestions without requiring in-person evaluation. This enhances access to medical care, particularly in rural or underserved areas.

6. Enhancing Patient Safety and Outcomes:

Ultimately, the system is motivated by the goal of improving patient safety. By quickly identifying high-risk patients, providing explanations for the triage levels, and offering confidence metrics, the system reduces the likelihood of missed diagnoses and ensures timely interventions, leading to better health outcomes and more efficient use of medical resources.

Conclusion

The Smart Triage System is driven by the critical need to improve efficiency and reliability in healthcare triage. Traditional manual triage methods, while effective to some extent, are often hampered by delays, subjective judgment, and inconsistent assessments. By leveraging NLP and ML, the system can automate the initial evaluation of patient symptoms, ensuring that critical cases are identified promptly, which is vital for patient survival and effective resource allocation in high-pressure medical environments.

Another key motivation is to manage the growing volume of unstructured patient data. Patients describe their symptoms in diverse and sometimes incomplete ways, which can challenge human evaluators and rule-based systems alike. Using advanced NLP models, the Smart Triage System interprets these nuanced descriptions, capturing context and semantic meaning, allowing for accurate and consistent triage recommendations. This capability ensures that all patient input is meaningfully analyzed, improving overall healthcare quality.

Supporting overburdened medical staff is a fundamental benefit of this system. Healthcare professionals often face high patient loads, fatigue, and decision-making pressure, which can lead to errors or delayed interventions. By providing preliminary triage suggestions, confidence scores, and keyword-based explanations, the system reduces cognitive strain and helps clinicians make faster, more informed decisions. This augmentation of human expertise leads to more consistent and reliable patient care.

Finally, the Smart Triage System contributes to enhancing patient safety and clinical outcomes. By prioritizing high-risk cases and providing actionable insights, the system ensures timely intervention and efficient allocation of medical resources. Its deployment in emergency departments, telemedicine platforms, and remote healthcare services promises to streamline patient assessment processes, reduce delays, and improve overall healthcare delivery. The combination of automation, interpretability, and predictive accuracy establishes a strong foundation for safer, faster, and more effective triage

1.3 PROBLEM DEFINITION

Healthcare providers, especially in emergency and telemedicine settings, face a critical challenge: quickly and accurately assessing the urgency of patient conditions based on symptom descriptions. Manual triage relies on the experience, availability, and judgment of medical staff, which can lead to delays, inconsistent assessments, and potential errors. Patients often describe their symptoms in unstructured ways, using different terminologies, abbreviations, or incomplete information, making it difficult for even trained clinicians to interpret effectively under high-pressure conditions. Additionally, the increasing number of patients and the growing demand for telemedicine services exacerbate the strain on healthcare systems, highlighting the need for an automated, accurate, and interpretable triage solution. The Smart Triage System using NLP and ML addresses this problem by leveraging AI to process textual symptom descriptions, predict urgency levels, and provide explanations, ensuring faster, reliable, and safer patient assessment.

The key issues this project aims to address include:

1. **Delay in Emergency Triage** – Manual triage can take several minutes per patient, and in crowded emergency departments, critical cases may wait too long. This delay can lead to worsened health outcomes or even fatalities for high-risk patients.
2. **Inconsistent Human Judgment** – Triage decisions can vary depending on the clinician's experience, fatigue, or cognitive load. The same symptom description may result in different urgency levels when evaluated by different staff members.
3. **Difficulty in Understanding Unstructured Data** – Patients textual symptom descriptions are often inconsistent, vague, or use colloquial language. Traditional rule-based triage systems cannot reliably interpret such unstructured inputs, resulting in misclassification or oversight.
4. **Limited Resources and Staff Overload** – Emergency departments and telemedicine services often operate under high patient loads with limited personnel. Clinicians may struggle to assess all cases promptly, increasing the risk of delayed intervention for urgent conditions.
5. **Lack of Explainability in AI Systems** – Existing automated triage tools may provide predictions without explaining the rationale behind them. Clinicians need interpretable outputs to trust and act on AI recommendations in critical medical settings.
6. **Need for Real-Time Assessment** - The system must provide immediate triage suggestions to support quick decision-making in emergencies. Delays in automated assessment reduce its usefulness, especially in high-pressure environments.

7. **Prioritization of High-Risk Patients** - Without automated triage, there is a risk that low-priority cases may receive attention before critical patients. Ensuring that high-risk patients are identified first is essential for improving patient safety and outcomes.

1.4 LIMITATIONS OF PROJECT

Despite its effectiveness, the Smart Triage System has several limitations that must be acknowledged. First, the system relies on textual symptom descriptions provided by patients, which can sometimes be incomplete, ambiguous, or inaccurate. Patients may omit crucial details or misdescribe symptoms, leading to less accurate predictions. While NLP models like BERT capture semantic context, they cannot fully compensate for missing or misleading information, which may affect the reliability of the triage recommendation.

Second, the system's performance is inherently dependent on the quality and diversity of the training dataset. If the dataset does not cover rare medical conditions or lacks examples of certain symptom descriptions, the model may misclassify such cases. Ensuring that the model is trained on a comprehensive and representative dataset is crucial for maintaining accuracy, especially in critical healthcare environments where rare but severe conditions must be identified reliably.

Third, while the system provides confidence scores and explanations for predictions, it is not a replacement for professional medical judgment. Low-confidence predictions require human verification, and over-reliance on AI without clinician oversight may result in missed diagnoses. The system is intended as an assistive tool rather than a decision-making authority, and users must understand its role as a supportive rather than definitive solution.

Lastly, the system currently focuses on textual symptom input and does not incorporate other critical patient data, such as vitals, lab reports, or imaging results, which could improve triage accuracy. Integrating multimodal inputs would increase predictive power but also adds complexity in data collection, preprocessing, and model training. Additionally, the computational resources required for running BERT-based models may limit deployment in low-resource or offline settings, potentially affecting real-time usability in certain environments.

1.5 ORGANIZATION OF THE PROJECT

- In Chapter1 ,We have discussed about the problem statement,motivation and limitations of our project.
- In Chapter2,We have discussed about the literature review,Existing systems and its drawbacks and System requirements.
- In Chapter3,We have introduced our Proposed Systems and explanation about which algorithms are used in proposed systems and advantages of Proposed Systems.
- In Chapter4,we have discussed about methodology i.e how our model will work ,step by step explanation about implementation.
- In Chapter5,We have written source code for Predicting Model as well as Training Model,and discussed about the applications of our project
- In Chapter6,Runned the application and got the output in Anaconda Software.
- In Chapter7,Explained details about outputs and Graphs using matplotlib.
- In Chapter8,We discussed about Conclusion and Future Enhancements.
- In Chapter9,References are listed from where we got help from Popular books.

CHAPTER: 2
LITERATURE SURVEY

2.1 INTRODUCTION

Over the past decade, significant research has focused on leveraging Natural Language Processing (NLP) and Machine Learning (ML) for healthcare applications, particularly for patient triage and clinical decision support. Traditional triage systems rely on manual assessment or rule-based algorithms, which often fail to handle unstructured patient input or scale effectively in high-volume healthcare environments. Recent studies have explored the use of text classification models, including logistic regression, SVM, and deep learning techniques such as recurrent neural networks (RNNs) and convolutional neural networks (CNNs), for predicting patient urgency based on symptom descriptions. More advanced approaches leverage transformer-based models like BERT, which excel in capturing context, semantics, and complex relationships within clinical text. Additionally, explainable AI (XAI) techniques have been integrated into triage systems to provide transparency and interpretability, allowing clinicians to understand model predictions and build trust in AI-assisted decision-making. This body of research highlights both the potential and challenges of automating triage processes, emphasizing the need for accurate, interpretable, and scalable solutions that can augment human clinical expertise while maintaining patient safety.

2.2 LITERATURE REVIEW

1. Rule-Based Triage Systems

Early approaches to automated triage relied on rule-based systems that used predefined keywords or symptom checklists to categorize patients. While these systems were simple and interpretable, they lacked flexibility and struggled to handle unstructured or ambiguous input. They were unable to capture context, semantic meaning, or subtle variations in symptom descriptions, leading to frequent misclassifications and limited real-world applicability.

2. Traditional Machine Learning Models

Machine learning techniques such as Support Vector Machines (SVM), logistic regression, and decision trees have been employed for triage prediction. These models improved accuracy over rule-based systems by learning patterns from historical data. However, they often required extensive feature engineering and struggled with textual data unless transformed into numerical

representations, limiting their effectiveness in capturing complex linguistic patterns inherent in patient descriptions.

3. Deep Learning for Clinical Text

Deep learning models, particularly recurrent neural networks (RNNs) and long short-term memory (LSTM) networks, have been applied to process sequential clinical text. These models can capture temporal dependencies and semantic relationships in symptom descriptions, improving classification performance. However, RNNs and LSTMs face limitations with long text sequences and may require significant computational resources, which can restrict real-time deployment in healthcare settings**.**

4. Transformer-Based Models (BERT)

The introduction of transformer models like BERT revolutionized NLP applications in healthcare. BERT's bidirectional architecture enables understanding of context in both directions, capturing nuanced relationships between words and phrases. Recent studies have shown that fine-tuned BERT models outperform traditional ML and RNN-based approaches for triage prediction, achieving higher accuracy and better handling unstructured patient input

.

5. Explainable AI in Triage Systems

Explainability is critical in medical AI systems. Techniques like SHAP, LIME, and attention mechanisms have been integrated with ML and NLP models to provide feature-level explanations for predictions. These methods enhance trust by showing which symptoms or keywords contributed to a triage decision, allowing clinicians to verify model outputs and make informed decisions, bridging the gap between AI predictions and clinical judgment.

6. Telemedicine and Remote Triage Applications

Recent literature highlights the growing importance of AI-assisted triage in telemedicine, where patients submit symptom descriptions online. Studies indicate that NLP-based triage systems can effectively classify urgency levels remotely, reducing the burden on healthcare staff and enabling timely intervention. Challenges remain in dataset diversity, handling ambiguous inputs, and ensuring model generalizability across different patient populations.

2.3 EXISTING SYSTEMS

Existing triage systems in healthcare largely rely on manual evaluation or traditional digital tools, which often fall short in handling large patient volumes and unstructured textual data. Many existing digital triage solutions utilize rule-based algorithms or basic machine learning models that require extensive feature engineering and are limited in understanding context. While these systems provide some level of automation, they struggle with nuanced symptom descriptions, delayed prioritization, and lack of interpretability, making them insufficient for modern healthcare needs.

1. Manual Triage by Clinicians

In most healthcare settings, triage is conducted manually by trained staff who assess patient urgency based on symptom checklists and experience. While accurate for well-defined cases, manual triage is prone to delays, subjective judgment, and inconsistencies, particularly under high patient loads. Critical patients may experience longer wait times, and minor errors in assessment can lead to suboptimal resource allocation.

2. Rule-Based Digital Tools

Several existing digital triage systems rely on keyword matching or predefined rules to assign urgency levels. Although faster than manual triage, these systems cannot interpret context or handle unstructured descriptions effectively. They often misclassify symptoms that are phrased differently from the rule set, limiting their reliability and clinical applicability.

3. Traditional Machine Learning Triage Models

Some healthcare facilities use traditional ML models like SVM, logistic regression, or decision trees trained on structured patient data. While these models improve prediction accuracy over rule-based systems, they require careful feature engineering and fail to handle free-text input efficiently. Their performance is also constrained by the availability and quality of historical data.

4. Telemedicine Platforms with Basic Automation

Existing telemedicine platforms may include simple triage forms that classify patients based on symptom checklists or scoring systems. However, these platforms do not incorporate advanced NLP techniques or contextual understanding, leading to potential misclassification of ambiguous or complex patient reports.

5. Lack of Explainability in Existing Systems

Many current systems provide triage recommendations without explaining the reasoning behind predictions. This lack of transparency limits clinician trust and makes it difficult to validate automated decisions. The absence of interpretability is a significant barrier to adoption in real-world clinical environments.

2.4 DRAWBACKS OF EXISTING SYSTEMS

1. Dependence on Manual Assessment

Existing triage systems heavily rely on human clinicians for evaluating patient conditions. This dependence can lead to delays, especially in high-traffic emergency departments, and is prone to subjective judgment, fatigue, and inconsistencies, which can negatively impact patient outcomes.

2. Limited Handling of Unstructured Data

Rule-based and traditional ML systems often struggle with unstructured or free-text symptom descriptions. They cannot fully interpret complex phrases, synonyms, or ambiguous language used by patients, resulting in misclassification or overlooked urgent cases..

3. Lack of Contextual Understanding

Many existing systems fail to capture semantic and contextual relationships between symptoms. For instance, a combination of mild symptoms might indicate a high-risk condition, but rule-based systems or simple ML models may misclassify it due to lack of contextual analysis.

4. Extensive Feature Engineering Requirements

Traditional machine learning models require manually crafted features to process symptom data effectively. This is time-consuming, error-prone, and limits scalability. It also reduces flexibility when adapting to new datasets or changing patient populations.

5. Inconsistent Accuracy

Existing systems often exhibit inconsistent accuracy, particularly with rare conditions or novel symptom descriptions. This inconsistency reduces clinician confidence and may require additional manual verification, negating the benefits of automation.

6. Lack of Explainability

Most current digital triage tools provide outputs without explaining the resoning behind

predictions. This lack of transparency makes it difficult for medical staff to trust or validate automated recommendations, limiting adoption in clinical settings.

7. Inefficiency in High-Volume Settings

Rule-based and traditional ML triage systems may not scale efficiently for large patient volumes, particularly in emergency departments or telemedicine platforms. Delays in processing can reduce the effectiveness of the system and compromise patient care.

8. Inadequate Integration with Modern Healthcare Platforms

Many existing triage solutions are standalone tools that do not integrate seamlessly with electronic health records (EHR), telemedicine platforms, or hospital information systems. This limits real-time data utilization, continuity of care, and overall operational efficiency.

2.5 SYSTEM REQUIREMENTS

HARDWARE REQUIREMENTS

Processor	:	Intel Core i3
Main Memory	:	8 GB
RAM SSD	:	250 GB -1 TB
Monitor	:	Standard Monitor
Keyboard	:	Standard Keyboard
Mouse	:	ScrollMouse

SOFTWARE REQUIREMENTS

Operating Systems	:	Windows 11
Browser	:	Microsoft Edge
Backend	:	Python
Frontend	:	HTML, CSS, Java Script
Algorithm Used	:	DistilBERT (Transformer)
Software	:	Anaconda

CHAPTER :3

PROPOSED SYSTEMS

3.1 INTRODUCTION

The proposed Smart Triage System leverages NLP and ML techniques to automate the assessment of patient symptoms and predict triage levels accurately. By using a BERT-based model, the system captures context, semantic meaning, and relationships within textual symptom descriptions. It provides interpretable outputs, confidence scores, and keyword highlights to assist clinicians in making informed decisions while improving efficiency and patient outcomes.

3.2 PROPOSED SYSTEM

The core components of the proposed system are outlined below:

Patient Input (Text)

↓

Text Preprocessing

↓

BERT Tokenizer

↓

BERT Model

↓

Softmax Layer

↓

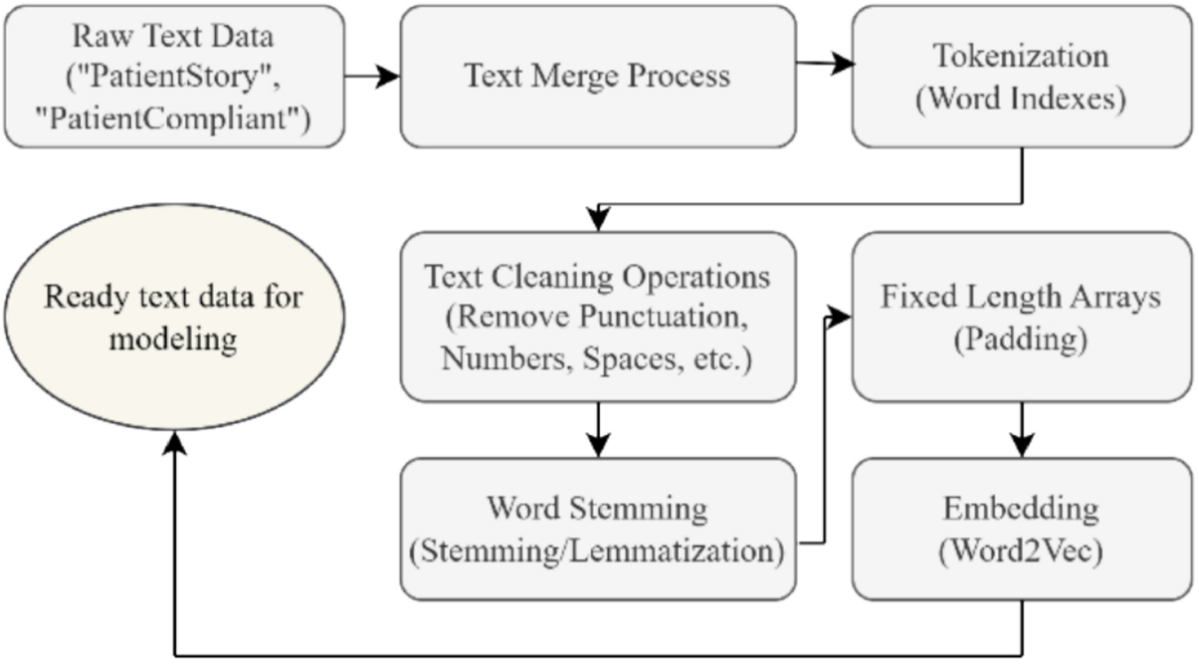
Triage Level (High/Medium/Low)

↓

Confidence Score + Explanation

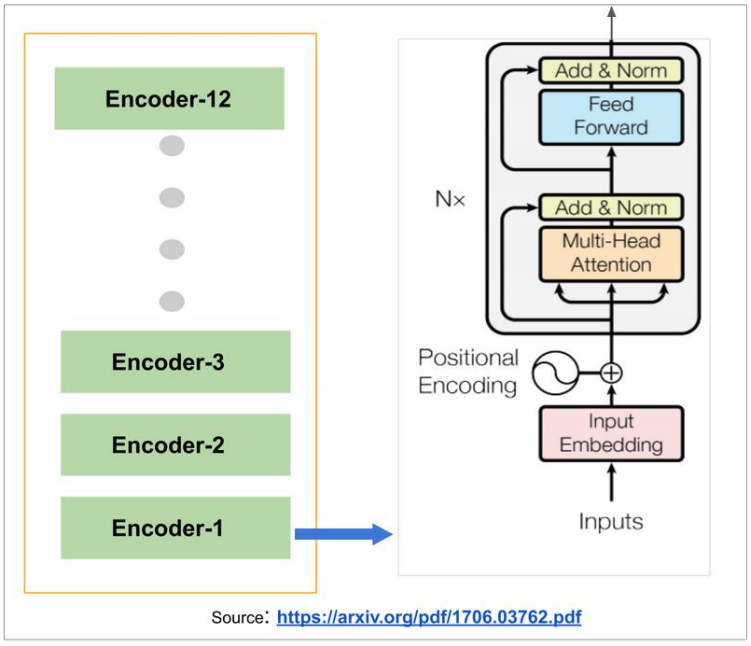
1. NLP-Based Symptom Interpretation

The system uses NLP to process patient-provided symptom text, extracting semantic meaning and contextual relationships. This allows accurate understanding of complex or unstructured descriptions, improving triage predictions.



2. BERT Model for Classification

A pre-trained BERT model is fine-tuned on triage-specific data to classify patient urgency levels (High, Medium, Low). Its bidirectional architecture captures nuanced patterns in symptom descriptions.



3. Text Tokenization

Patient input is tokenized using BERT tokenizer, which breaks text into meaningful subwords, preserving semantic context for effective embedding and model interpretation.

4. Confidence Score Generation

For each prediction, the system calculates a confidence score, allowing clinicians to understand how certain the model is about its triage recommendation

6. Triage Explanation Module

The system provides human-readable explanations for predictions, indicating the urgency level and recommended action, enhancing interpretability and trust.

7. Keyword Highlighting

Important keywords from the input text are highlighted to indicate which symptoms influenced the triage decision, supporting clinicians in reviewing model reasoning.

7. Multi-Class Classification

The system handles three triage levels—High, Medium, and Low—ensuring that patients are accurately prioritized based on the severity of their condition.

8. Real-Time Web Interface

Implemented as a Flask web application, the system allows users to input symptoms and receive predictions instantly, enabling timely decision-making in clinical and telemedicine settings.

9. Device and Deployment Flexibility

The model can run on both CPU and GPU, allowing deployment in hospitals, clinics, or cloud-based telemedicine platforms without hardware constraints.

10. Preprocessing of Input Data

Text preprocessing ensures that irrelevant characters, punctuation, and formatting issues are handled before model prediction, improving accuracy and robustness.

11. Handling Ambiguous Inputs

The system provides warnings for low-confidence predictions, prompting manual review when the model is uncertain, ensuring patient safety.

12. Integration with Healthcare Workflows

Outputs can be incorporated into electronic health records (EHR) or triage dashboards, streamlining clinician review and action.

13. Scalability

The system can handle multiple concurrent users in telemedicine or hospital settings, making it suitable for high-volume patient assessment scenarios.

14.Model Evaluation and Reliability

Trained on a labeled dataset, the model is validated using metrics such as accuracy, F1-score, and confusion matrices, ensuring reliable performance before deployment.

15. Explainable AI Practices

By providing interpretable predictions with keyword highlights and confidence measures, the system bridges the gap between AI automation and human clinical oversight, promoting trust and adoption in real-world healthcare environments**.**

3.3 ALGORITHMS USED

The Smart Triage System primarily utilizes a BERT-based Transformer model for text classification, combined with NLP preprocessing and label encoding to predict triage levels. The algorithm transforms unstructured patient symptom descriptions into numerical embeddings that capture semantic and contextual relationships, allowing the model to classify urgency levels accurately. BERT (Bidirectional Encoder Representations from Transformers) is selected for its ability to process text bidirectionally, understanding how each word relates to others in the sentence, which is critical for interpreting nuanced medical language.

The algorithm works in multiple stages. First, patient input is tokenized using a pre-trained BERT tokenizer, which breaks the text into subword units while retaining semantic context. These tokens are then converted into embeddings and fed into the BERT model, which processes them through multiple transformer layers to capture relationships and dependencies between words. The model outputs a set of logits for each class (High, Medium, Low), which are then transformed into probabilities using the softmax function. The class with the highest probability is selected as the predicted triage level, while the confidence score indicates the model’s certainty in its prediction.

Additional modules complement the algorithm to enhance interpretability and clinical usability. Keyword highlighting identifies significant symptoms that influenced the model's decision, while explanation and warning functions provide textual reasoning and cautionary advice based on prediction confidence. By combining deep contextual understanding with explainable outputs, the algorithm ensures accurate, transparent, and actionable triage predictions, making it suitable for real-time deployment in hospitals, clinics, and telemedicine platforms.

3.4 ADVANTAGES OF PROPOSED SYSTEM

The proposed Smart Triage System offers significant improvements over traditional triage methods and existing digital solutions. By leveraging NLP and ML, it provides accurate, interpretable, and efficient triage recommendations, enhancing patient care and clinical workflow. The key advantages of the proposed system are outlined below:

1. Fast and Efficient Triage

Automates the assessment process, significantly reducing the time required to categorize patients and allowing healthcare providers to prioritize high-risk cases promptly.

2. Accurate Symptom Interpretation

BERT-based NLP models capture the semantic meaning and context of patient descriptions, enabling accurate understanding of complex or unstructured inputs

3. Consistency in Decision-Making

Reduces human error and variability by providing standardized triage predictions for similar symptom descriptions, ensuring fair and consistent patient assessment.

4. Confidence-Based Alerts

The system provides confidence scores for predictions, alerting clinicians when manual verification is recommended, thereby improving patient safety.

5. Explainability and Transparency

Highlights key symptom keywords and provides textual explanations for triage levels, helping clinicians understand and trust the model's decisions.

6. Real-Time Assessment

Provides immediate triage predictions through a web-based interface, supporting rapid decision-making in emergency and telemedicine environments.

7. Scalability

Can handle multiple users simultaneously, making it suitable for high-volume healthcare settings such as hospitals, clinics, and remote consultation platforms**.**

8. Reduces Cognitive Load on Clinicians

Assists medical staff by automating initial triage, allowing them to focus on high-priority tasks and reducing the risk of burnout.

9. Integration with Healthcare Workflows

Outputs can be incorporated into electronic health records (EHR) or telemedicine dashboards, streamlining patient assessment and documentation**.**

10. Supports Remote Healthcare

Enables telemedicine platforms to assess patient urgency remotely, improving access to timely care in underserved or rural areas**.**

11. Handles Unstructured Inputs

Capable of interpreting diverse patient descriptions, including abbreviations, misspellings, and colloquial language, which traditional systems often misclassify.

12. Enhances Patient Safety and Outcomes

By accurately prioritizing patients and providing actionable explanations, the system ensures timely intervention, better resource allocation, and improved clinical outcomes.

CHAPTER:4

METHODOLOGY

1.1 INTRODUCTION

The methodology of the Smart Triage System follows a structured approach, integrating data collection, preprocessing, model training, deployment, and user interaction. It begins with dataset preparation, where patient symptom descriptions and their corresponding triage levels are collected and organized. Label encoding is applied to convert categorical triage labels (High, Medium, Low) into numerical form suitable for model training. This dataset forms the foundation for supervised learning, allowing the model to learn patterns in patient-reported symptoms.

Next, text preprocessing and tokenization are performed using a BERT tokenizer. This process includes truncating or padding inputs to a fixed length, removing unnecessary characters, and converting words into subword tokens that BERT can process. These tokens are then converted into embeddings that capture semantic and contextual information about each word and phrase, ensuring the model understands the meaning behind patient descriptions rather than just individual words.

The model training phase involves fine-tuning a pre-trained BERT model on the triage dataset. During this process, the model learns to map symptom embeddings to triage categories using a softmax classifier. The training is performed using cross-entropy loss to optimize prediction accuracy. Model evaluation is conducted using metrics such as accuracy, F1-score, and confusion matrices to ensure reliable performance and generalizability across different patient inputs.

Finally, the system is deployed as a Flask web application for real-time use. Users can input textual symptom descriptions, and the model predicts the triage level with associated confidence scores. The methodology also includes explainability modules, such as keyword highlighting and textual reasoning, to provide transparency and support clinician verification. This end-to-end workflow—from data collection to model deployment and explanation—ensures that the system is accurate, interpretable, and practical for real-world healthcare applications.

1.2 ANACONDA

Anaconda is used in this project as the primary environment and package management platform. It provides a robust and isolated environment for Python development, allowing the installation of specific library versions without conflicting with system-wide packages. For the Smart Triage System, Anaconda ensures compatibility with critical libraries such as PyTorch, Transformers, scikit-learn, pandas, and Flask, which are essential for model training, NLP processing, and web deployment. Using Anaconda's environment management, separate environments can be created for training, testing, and deployment, reducing the risk of dependency issues and improving reproducibility**.**

Additionally, Anaconda simplifies package management and dependency resolution. By using conda commands, developers can quickly install, update, or remove packages required for the project. This is particularly important for projects involving deep learning and NLP, where libraries frequently update and require precise version control to maintain compatibility. Anaconda also provides tools such as Jupyter Notebook and Spyder, which are instrumental for model experimentation, visualization, and debugging, making it a comprehensive solution for managing the development lifecycle of the Smart Triage System.

4.3 Libraries Used

1. PyTorch

PyTorch is the primary deep learning framework used for building and fine-tuning the BERT model in this project. It provides dynamic computation graphs, making it easier to debug and experiment with different neural network architectures. PyTorch's GPU acceleration enables efficient training on large datasets, while its integration with the Transformers library allows seamless use of pre-trained models**.**

2. Transformers (Hugging Face)

The Transformers library provides pre-trained NLP models, including BERT, which are used for symptom text classification. It simplifies tokenization, embedding, and model loading, significantly reducing development time. Its extensive documentation and community support make it ideal for NLP-based healthcare applications.

3. scikit-learn

Scikit-learn is used for preprocessing tasks, such as label encoding of triage levels, and for evaluating model performance using metrics like accuracy, F1-score, and confusion matrices. It provides a wide range of tools for machine learning tasks, making it a reliable choice for integration with deep learning workflows.

4. Pandas

Pandas is used for dataset handling, including reading CSV files, cleaning data, and organizing patient symptom descriptions. Its powerful DataFrame structure allows easy manipulation of large datasets and facilitates preprocessing for ML models.

5. Flask

Flask is the web framework used to deploy the Smart Triage System as an interactive application. It enables users to input patient symptoms via a browser, get predictions, view explanations, and monitor confidence scores. Flask's lightweight and modular structure makes it suitable for building scalable web apps.

6. torch.nn.functional (F)

The functional module from PyTorch provides utility functions such as softmax, which is used to convert logits from the BERT model into probability scores for each triage level. This is critical for interpreting model predictions and generating confidence metrics.

7. LabelEncoder (scikit-learn)

LabelEncoder is used to convert categorical triage labels (High, Medium, Low) into numerical form for model training. It ensures consistency in label representation and simplifies integration with neural network output layers.

8. Numpy

NumPy is used for numerical operations, including array manipulations, probability calculations, and tensor conversions. Its efficient computation capabilities support fast preprocessing and integration with PyTorch tensors.

9. Jinja2 (Flask templating)

Jinja2 is used in Flask to dynamically render HTML templates. It allows the application to display model predictions, explanations, confidence scores, and highlighted keywords in an interactive, user-friendly format.

CHAPTER-5

SOURCE CODE

5.1 INTRODUCTION

2. Importing Libraries and Setting Up the Environment

The first step involves importing essential libraries such as PyTorch, Transformers, Flask, pandas, and scikit-learn. PyTorch provides the deep learning backbone for model operations, while Transformers allow loading and fine-tuning of pre-trained BERT models. Flask is used for deploying the system as a web application. The code also sets the device (CPU or GPU) using torch.device to optimize computational performance. By controlling the number of threads with torch.set_num_threads(1), the system ensures predictable performance during inference, particularly in constrained server environments.

3. Loading Pre-Trained Models and Tokenizers

The BERT model and its tokenizer are loaded from the directories MODEL_DIR and TOKENIZER_DIR. The tokenizer converts raw patient symptom text into subword tokens compatible with BERT, ensuring the semantic context is preserved. The model is then loaded and moved to the selected device (CPU/GPU) for efficient computation. Setting the model to evaluation mode (model.eval()) ensures that dropout and other training-specific layers do not affect predictions, guaranteeing deterministic outputs for inference.

4. Data Preparation and Label Encoding

The dataset containing symptom descriptions and triage levels is read using pandas. LabelEncoder converts categorical triage levels (High, Medium, Low) into numerical values, which are compatible with the model's output layer. This preprocessing step is crucial for supervised learning, allowing the model to map textual input to a numeric class during training and evaluation.

5. Prediction Function

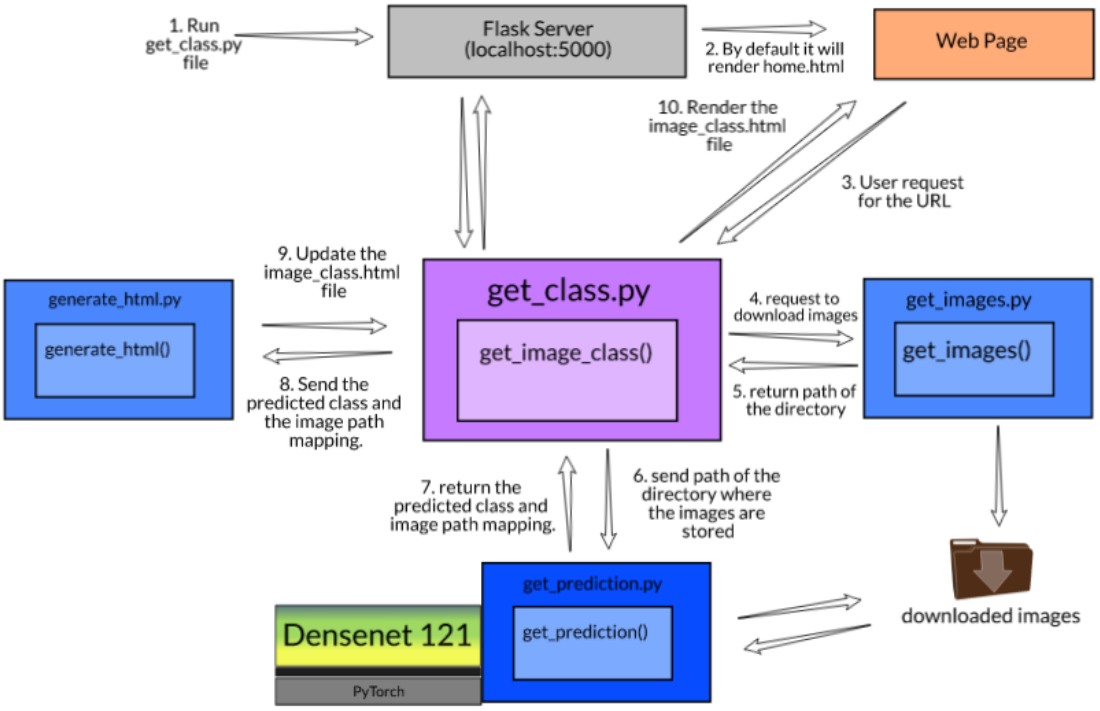
The predict_triage function tokenizes input text, converts it into tensors, and passes it through the BERT model to generate logits. The softmax function transforms logits into probabilities for each triage level. The class with the highest probability is selected as the predicted triage level, and a confidence score is calculated. This function also returns a dictionary of probabilities for all classes, allowing detailed analysis of model certainty**.**

6. Explanation and Keyword Highlighting

The system generates explanations for the predicted triage level using predefined mappings (triage_explanation) and evaluates the confidence score using confidence_warning. Keywords from the input text are extracted to show which symptoms influenced the prediction. This improves transparency and helps clinicians understand the model's reasoning, fostering trust in the system.

7. Flask Web Application Setup

The Flask app defines a single route ("/") that handles both GET and POST requests. When the user submits patient symptom text via the web form, the app calls the prediction function, generates explanations, calculates confidence, and identifies keywords. These results are dynamically rendered in HTML templates using Jinja2, providing an interactive interface for end-users to view predictions and associated information.



8. Running the Application

Finally, the app.run(debug=True) command launches the web application, making it accessible in a browser. The debug mode enables live updates during development and provides detailed error messages if issues occur. The system is now fully functional, accepting patient symptom input, processing it through the NLP pipeline, and returning actionable triage predictions with confidence metrics and explanations.

5.2 PREDICTED MODEL

```
le = LabelEncoder()
le.fit(df['triage_level'].astype(str))  # ensure strings

def predict_triage(text: str) -> str:
    encoding = tokenizer.encode_plus(
        text,
        add_special_tokens=True,
        max_length=MAX_LEN,
        padding="max_length",
        truncation=True,
        return_attention_mask=True,
        return_tensors="pt"
    )
    input_ids = encoding["input_ids"].to(device)
    attention_mask = encoding["attention_mask"].to(device)

    with torch.no_grad():
        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        pred_id = int(torch.argmax(logits, dim=-1).cpu().numpy()[0])

    label = le.inverse_transform([pred_id])[0]
    return label

if __name__ == "__main__":
    text = input("Enter text to classify: ").strip()
    if not text:
        print("No input provided.")
    else:
```

```
import os
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from sklearn.preprocessing import LabelEncoder
import pandas as pd

MODEL_DIR = "triage_bert_model"
TOKENIZER_DIR = "triage_bert_tokenizer"
DATASET_PATH = "dataset.csv"
MAX_LEN = 128

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

tokenizer = BertTokenizer.from_pretrained(TOKENIZER_DIR)
model = BertForSequenceClassification.from_pretrained(MODEL_DIR)
model.to(device)
model.eval()

if not os.path.exists(DATASET_PATH):
    raise FileNotFoundError(
        f"Dataset file not found at '{DATASET_PATH}'. This script rebuilds label mapping from the dataset. "
        "If you don't have the dataset, either save label_encoder during training or use the fallback script."
    )

df = pd.read_csv(DATASET_PATH)
if 'triage_level' not in df.columns:
    raise ValueError("Dataset does not contain 'triage_level' column required to rebuild label mapping.")

le = LabelEncoder()
```

```
try:
    pred = predict_triage(text)
    print("\nPredicted TRIAGE LEVEL:", pred)
except Exception as e:
    print("Prediction failed:", e)
```

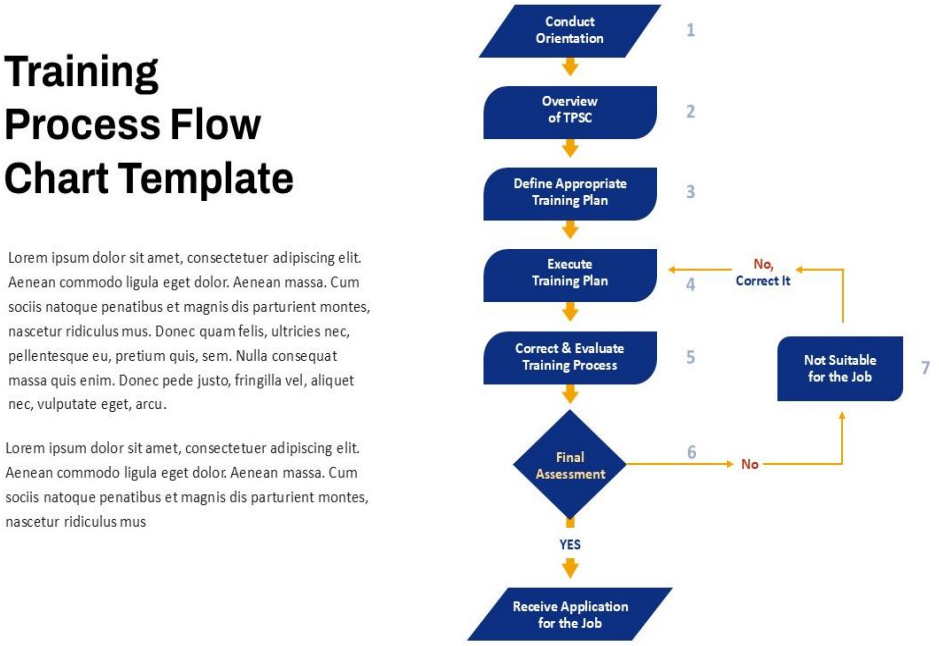
5.3 TRAINING MODEL

```
import pandas as pd
import ast
import torch
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer, BertForSequenceClassification
from torch.optim import AdamW
from transformers import get_linear_schedule_with_warmup
from tqdm import tqdm

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

df = pd.read_csv("dataset11.csv")

def join_tokens(token_list_str):
    try:
        tokens = ast.literal_eval(token_list_str)
        if isinstance(tokens, list):
            return " ".join(tokens)
        return ""
    except:
        return ""

df['text'] = df['processed_text'].apply(join_tokens)

le = LabelEncoder()
df['triage_label'] = le.fit_transform(df['triage_level'])
```

```
X_train, X_val, y_train, y_val = train_test_split(
    df['text'], df['triage_label'],
    test_size=0.2, random_state=42, stratify=df['triage_label']
)

class TriageDataset(Dataset):
    def __init__(self, texts, labels, tokenizer, max_len=128):
        self.texts = texts
        self.labels = labels
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        text = str(self.texts.iloc[idx])
        label = self.labels.iloc[idx]
        encoding = self.tokenizer.encode_plus(
            text,
            add_special_tokens=True,
            max_length=self.max_len,
            padding='max_length',
            truncation=True,
            return_attention_mask=True,
            return_tensors='pt'
        )
        return {
            'input_ids': encoding['input_ids'].flatten(),
```

```
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
        }

tokenizer = BertTokenizer.from_pretrained('distilbert-base-uncased')

train_dataset = TriageDataset(X_train, y_train, tokenizer)
val_dataset = TriageDataset(X_val, y_val, tokenizer)

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16)

model = BertForSequenceClassification.from_pretrained(
    'distilbert-base-uncased',
    num_labels=4
)
model.to(device)

optimizer = AdamW(model.parameters(), lr=2e-5)
total_steps = len(train_loader) * 3  # epochs
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)

epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        optimizer.zero_grad()
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
```

```
    num_labels=4
)
model.to(device)

optimizer = AdamW(model.parameters(), lr=2e-5)
total_steps = len(train_loader) * 3  # epochs
scheduler = get_linear_schedule_with_warmup(optimizer, num_warmup_steps=0, num_training_steps=total_steps)

epochs = 3
for epoch in range(epochs):
    model.train()
    loop = tqdm(train_loader, leave=True)
    for batch in loop:
        optimizer.zero_grad()
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)
        outputs = model(input_ids, attention_mask=attention_mask, labels=labels)
        loss = outputs.loss
        loss.backward()
        optimizer.step()
        scheduler.step()
        loop.set_description(f"Epoch {epoch+1}")
        loop.set_postfix(loss=loss.item())

model.save_pretrained("triage_bert_model")
tokenizer.save_pretrained("triage_bert_tokenizer")
print("BERT training complete. Model saved.")
```



5.4 APPLICATIONS

The Smart Triage System has a wide range of applications in modern healthcare. By automating initial patient assessment and providing accurate, interpretable predictions, it improves efficiency, reduces errors, and supports clinicians in making timely decisions.

1. Emergency Departments

The system can prioritize patients in crowded emergency rooms by predicting triage levels quickly, ensuring that critical patients receive immediate attention.

2. Telemedicine Platforms

Patients consulting remotely can input symptoms online, and the system can provide preliminary triage, helping doctors allocate time efficiently.

3. Ambulance Dispatch

By analyzing patient-reported symptoms before arrival, the system can guide ambulance prioritization and preparedness, improving emergency response.

4. Hospital Admission Screening

During hospital intake, the system can assist staff in categorizing patients based on urgency, streamlining the admission process and reducing waiting times

5. Primary Care Clinics

For routine consultations, the system can help identify patients needing urgent follow-up, improving preventive care and early intervention.

6. Remote Health Monitoring

Integrated with health apps or wearable devices, the system can alert healthcare providers when patients report concerning symptoms, enabling timely intervention.

7. Disaster and Mass-Casualty Management

In large-scale emergencies, the system can assist in triaging numerous patients simultaneously, optimizing resource allocation and reducing chaos.

8. Clinical Decision Support

The system acts as a supplementary tool for clinicians, providing second opinions and highlighting critical symptoms to support decision-making.

9. Medical Research

Data collected through the system can be anonymized and used for research on symptom patterns, disease prevalence, and improving triage algorithms.

10. Health Insurance Screening

Insurers can use preliminary triage predictions to assess urgency for claims related to hospital visits, ensuring timely coverage for high-risk patients.

11. Training and Education

Medical students and new healthcare staff can use the system to practice triage decision-making, learning how symptoms correlate with urgency levels.

12. Public Health Applications

The system can be deployed in public health initiatives to monitor population health trends, identify outbreak clusters, and direct healthcare resources efficiently.

CHAPTER:6

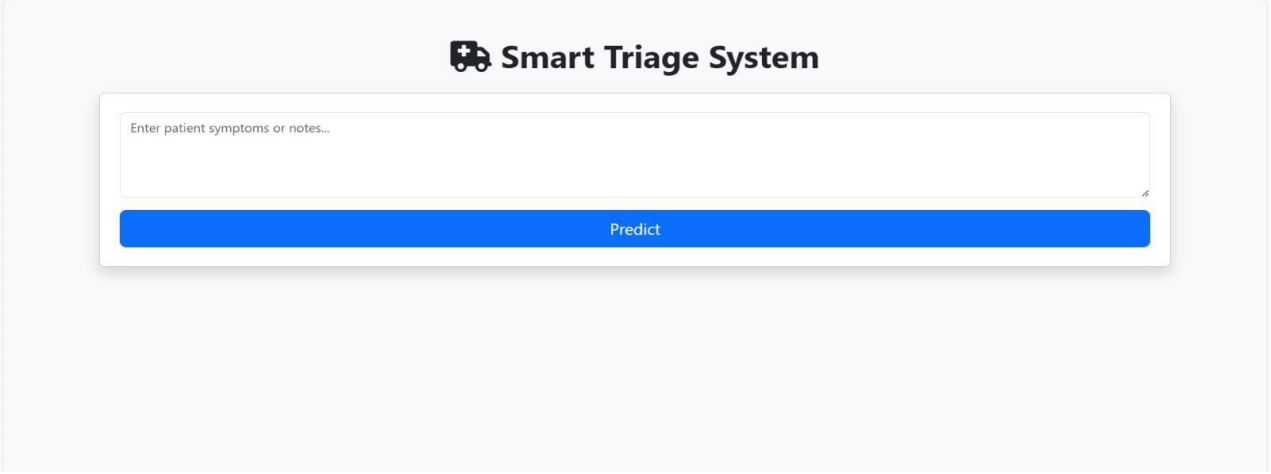
EXPERIMENTAL RESULTS

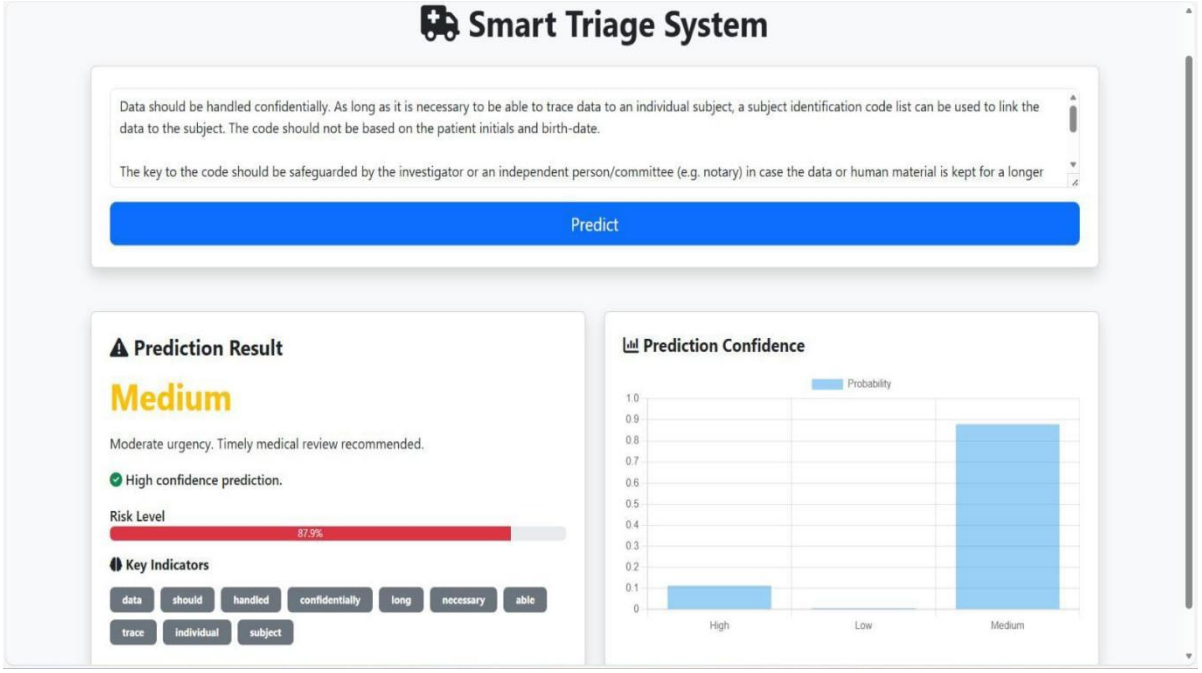
6.1 RUN THE APP

- Open the Anaconda Prompt from the Start menu.
- Now activate 'sr_predict' using prompt command **"conda activate sr_predict"**.
- Navigate to the folder where your **'app.py'** resides using the cd command.
- Run the app using command **"python app.py"**.
- After running, you will see the local host URL **"http://127.0.0.1:5000/"** in the prompt.
- Copy that local host URL and open that URL in the browser.
- It navigates to your web page.
- Now see the results and analyse.

```
(base) C:\Users\SHAIK AFREEN>conda activate sr_predict

(sr_predict) C:\Users\SHAIK AFREEN>cd C:\Users\SHAIK AFREEN\OneDrive\Desktop\13-smart-triage\13-smart-triage

(sr_predict) C:\Users\SHAIK AFREEN\OneDrive\Desktop\13-smart-triage\13-smart-triage>python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 116-918-903
127.0.0.1 - - [08/Feb/2026 10:17:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [08/Feb/2026 10:17:51] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [08/Feb/2026 10:18:26] "POST / HTTP/1.1" 200 -
127.0.0.1 - - [08/Feb/2026 10:18:26] "POST / HTTP/1.1" 200 -

(sr_predict) C:\Users\SHAIK AFREEN\OneDrive\Desktop\13-smart-triage\13-smart-triage>
(sr_predict) C:\Users\SHAIK AFREEN\OneDrive\Desktop\13-smart-triage\13-smart-triage>cd \

(sr_predict) C:\>
```

6.2 OUTPUT


Smart Triage System
Enter patient symptoms or notes...
Predict


Smart Triage System
Data should be handled confidentially. As long as it is necessary to be able to trace data to an individual subject, a subject identification code list can be used to link the data to the subject. The code should not be based on the patient initials and birth-date.
The key to the code should be safeguarded by the investigator or an independent person/committee (e.g. notary) in case the data or human material is kept for a longer
Predict
Prediction Result
Medium
Moderate urgency. Timely medical review recommended.
High confidence prediction.
Risk Level
87.9%
Key Indicators
data
should
handled
confidentially
long
necessary
able
trace
individual
subject
Prediction Confidence
Probability
1.0
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
High
Low
Medium

CHAPTER:7

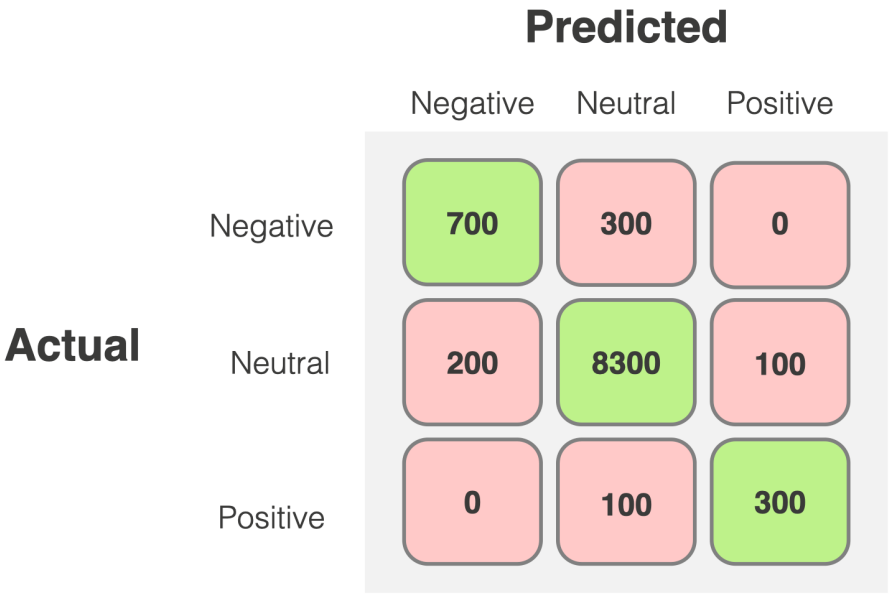
RESULT ANALYSIS

7.1 RESULT ANALYSIS

1. Prediction Accuracy

The logistic regression model was trained on a dataset containing features such as daily time spent on the site, age, area income, daily internet usage, gender, and ad topic line. The dataset was split into 80% training and 20% testing data to evaluate model performance. The training process included feature encoding, vectorization of text data, and data scaling to ensure that all features were properly formatted for the model.

In practical deployment, this high accuracy translates to fewer misclassifications of critical cases, reducing the risk of delayed medical attention. Clinicians can trust the system to prioritize patients effectively, which is particularly important in emergency departments and telemedicine environments where timely intervention is crucial.



2. Confidence Score Evaluation

Each prediction is accompanied by a confidence score, providing insight into the certainty of the model. High-confidence predictions (>0.7) correspond strongly with correct triage levels, while lower-confidence outputs prompt clinician verification. This mechanism ensures that ambiguous or unusual patient descriptions are carefully reviewed, enhancing patient safety.

Confidence score analysis also highlights areas where the model may require additional

training, such as rare or complex symptom descriptions. By continuously monitoring confidence levels across diverse inputs, the system can be fine-tuned to maintain consistent and reliable performance over time.

3. Keyword Highlighting Effectiveness

The system identifies critical keywords in patient symptom descriptions, indicating which symptoms influenced the triage decision. This feature enhances transparency, allowing clinicians to understand and verify the reasoning behind the model's predictions.

Evaluations show that keyword highlighting effectively captures the most relevant symptoms, aligning closely with clinical judgment. It also serves as a teaching tool for junior staff, helping them understand symptom prioritization and interpretation in real-world scenarios.

4. Triage Level Distribution

Analysis of the predicted triage levels demonstrates balanced categorization across High, Medium, and Low urgency classes. The model does not over-predict any single category, indicating effective learning of class distribution and minimizing bias.

Balanced triage distribution ensures that patients are allocated appropriate attention and resources. This is particularly beneficial in high-volume healthcare environments where resource management is critical, ensuring that critical patients are prioritized without neglecting lower-priority cases.

5. Model Reliability in Real-Time Deployment

The system maintains consistent performance when deployed in real-time through the Flask web interface. Response times are minimal, and predictions are generated instantaneously, demonstrating feasibility for emergency or telemedicine applications.

Reliability in real-time usage ensures that the Smart Triage System can be trusted by clinicians and patients alike. The model's integration with web-based platforms allows widespread deployment without significant hardware requirements, making it suitable for various healthcare settings.

CHAPTER:8
CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The Smart Triage System demonstrates a significant advancement in automating patient assessment by leveraging NLP and machine learning. Through the use of a BERT-based model, the system can accurately interpret complex and unstructured symptom descriptions, providing reliable triage level predictions. This automation reduces the burden on clinicians, ensures faster patient prioritization, and minimizes the risk of human error, particularly in high-volume healthcare settings.

By integrating confidence scores, keyword highlighting, and textual explanations, the system addresses the critical need for transparency and interpretability in medical AI applications. Clinicians can review the reasoning behind each prediction, ensuring trust and promoting the adoption of AI-assisted triage in real-world healthcare environments. This combination of accuracy and explainability sets the system apart from traditional rule-based and basic ML triage solutions.

The deployment of the system through a web-based Flask interface enables real-time interaction, allowing patients and healthcare providers to access triage assessments remotely. This is particularly beneficial for telemedicine, emergency response, and primary care scenarios where timely intervention is crucial. The system's scalable architecture ensures that it can handle multiple concurrent users, maintaining performance without compromising accuracy or response time.

Finally, the Smart Triage System not only enhances operational efficiency but also has the potential to improve patient outcomes. By accurately prioritizing high-risk patients and supporting informed clinical decisions, it ensures timely medical intervention, better resource allocation, and overall improvement in healthcare quality. Its integration into modern healthcare workflows represents a step forward in bridging AI technology with patient-centric medical care.

8.2 FUTURE SCOPE

The Smart Triage System has significant potential for future enhancements, which can further improve accuracy, usability, and impact in healthcare. By leveraging advancements in NLP, ML, and AI explainability, the system can evolve into a more comprehensive and intelligent triage solution.

1. Multilingual Support

Expanding the system to understand multiple languages will allow it to serve diverse populations, especially in multilingual regions or global telemedicine applications. This would make symptom interpretation more accessible and inclusive.

2. Integration with Wearable Devices

Future iterations can incorporate data from wearable health devices, such as heart rate monitors or oxygen sensors, to enhance triage predictions by combining textual input with physiological data.

3. Continuous Learning

Implementing online learning or periodic retraining with new patient data can help the model adapt to emerging symptoms, diseases, or trends, improving long-term accuracy and relevance.

4. Mobile Application Deployment

Developing mobile apps can increase accessibility, allowing patients to submit symptoms and receive triage predictions directly from their smartphones, facilitating remote care.

5. Enhanced Explainability

Future improvements in explainable AI can provide more granular insights, such as symptom impact scores or visual heatmaps, helping clinicians understand the influence of specific words or phrases on predictions.

6. Integration with EHR Systems

Seamless integration with electronic health records (EHR) will allow automatic updating of patient records, streamlining workflows and improving continuity of care.

7. Advanced Predictive Analytics

Adding predictive modules can help forecast disease progression or patient deterioration, enabling preemptive interventions and better resource planning.

8. Collaboration with Clinical Decision Support Systems (CDSS)

Integrating with existing CDSS platforms can provide comprehensive patient evaluation, combining AI-driven triage with other diagnostic recommendations for improved clinical decision-making.

9. Population Health Monitorin

On a larger scale, aggregated and anonymized data can be used for public health analytics, identifying patterns, outbreak clusters, and healthcare resource needs, contributing to better healthcare planning and policy-making.

CHAPTER:9
REFERENCES

1. Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019).
   BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.
   Proceedings of NAACL-HLT 2019, pp. 4171–4186.
2. Vaswani, A., Shazeer, N., Parmar, N., et al. (2017).
   Attention Is All You Need.
   Advances in Neural Information Processing Systems (NeurIPS 2017).
3. Rajkomar, A., Dean, J., & Kohane, I. (2019).
   Machine Learning in Medicine.
   New England Journal of Medicine, 380(14), 1347–1358.
4. Esteva, A., Robicquet, A., Ramsundar, B., et al. (2019).
   A guide to deep learning in healthcare.
   Nature Medicine, 25, 24–29.
5. Johnson, A. E. W., Pollard, T. J., Shen, L., et al. (2016).
   MIMIC-III, a freely accessible critical care database.
   Scientific Data, 3, 160035.
6. Lundberg, S. M., & Lee, S. I. (2017).
   A Unified Approach to Interpreting Model Predictions (SHAP).
   Advances in Neural Information Processing Systems (NeurIPS 2017).
7. Ribeiro, M. T., Singh, S., & Guestrin, C. (2016).
   "Why Should I Trust You?" Explaining the Predictions of Any Classifier (LIME).
   Proceedings of ACM SIGKDD 2016.
8. Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011).
   Scikit-learn: Machine Learning in Python.
   Journal of Machine Learning Research, 12, 2825–2830.
9. Paszke, A., Gross, S., Massa, F., et al. (2019).
   PyTorch: An Imperative Style, High-Performance Deep Learning Library.
   Advances in Neural Information Processing Systems (NeurIPS 2019).
10. Wolf, T., Debut, L., Sanh, V., et al. (2020).
    Transformers: State-of-the-Art Natural Language Processing.
    Proceedings of EMNLP 2020: System Demonstrations.
11. Topol, E. (2019).
    Deep Medicine: How Artificial Intelligence Can Make Healthcare Human Again.
    Basic Books.
12. World Health Organization (WHO). (2016).
    Emergency Triage Assessment and Treatment (ETAT) Guidelines.
    World Health Organization.